

Brand

一生践行韦特莱法则，一生保持高执行力高效率...

博客园 首页

Redis系列5：深入分析Cluster 集群模式

- Redis系列1：深刻理解高性能Redis的本质
- Redis系列2：数据持久化提高可用性
- Redis系列3：高可用之主从架构
- Redis系列4：高可用之Sentinel(哨兵模式)

1 背景

前面我们学习了Redis高可用的两种架构模式：主从模式、哨兵模式。解决了我们在Redis实例发生故障时，具备主从自动切换、故障转移的能力，最终保证服务的高可用。但是这些其实远远不够，随着我们业务规模的不断扩展，用户量膨胀，并发量持续提升。原有的主从架构，已经远远达不到我们一些问题出现，比如：

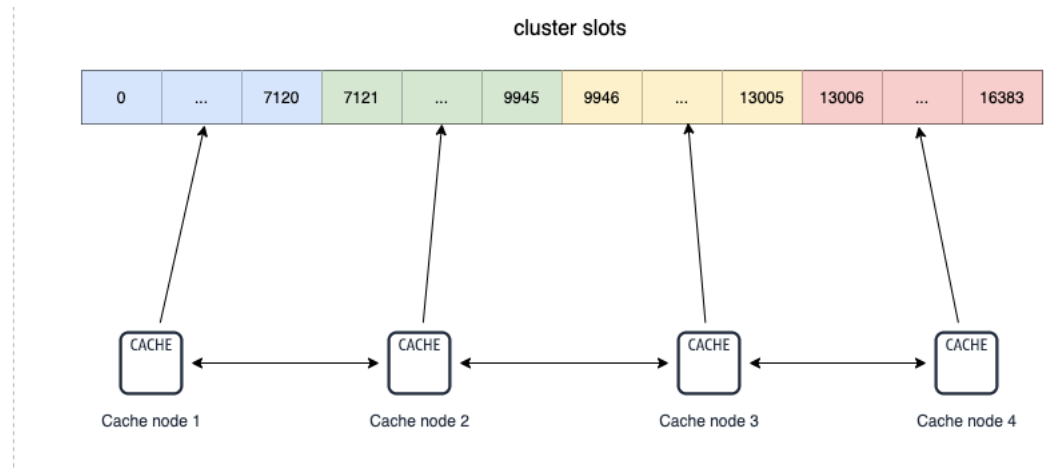
- 单机的CPU、内存、连接数、计算力都是有极限的，不能无限制的承载流量的扩增。
- 超额的请求、大规模的数据计算，导致必然的慢响应。

这时候就需要适当的推进架构的演进，来满足发展的需要。

2 Cluster 模式介绍

2.1 什么是Cluster模式

Cluster 即 集群模式，类似MySQL，Redis 集群也是一种分布式数据库方案，集群通过分片（sharding）模式来对数据进行管理，制、故障转移和流量调度的能力。这种 分治模式很常见，我们在 微服务系列:拆分策略 和 MySQL系列:分库分表 中实践过很多次Redis集群的做法是 将数据划分为 16384（2的14次方）个哈希槽（slots），如果你有多个实例节点，那么每个实例节点将管理其的信息会存储在各自所归属的节点中。以下图为例，该集群有4个 Redis 节点，每个节点负责集群中的一部分数据，数据量可以以例节点可以多分担一些压力。



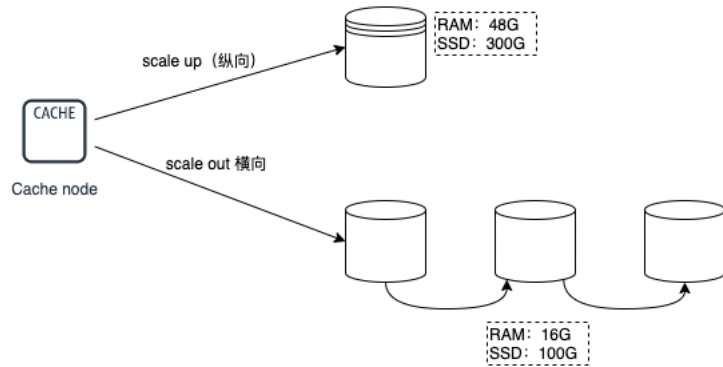
一个Redis集群一共有16384个哈希槽，你可以有1 ~ n个节点来分配这些哈希槽，可以不均匀分配，每个节点可以处理0个 到至多当16384个哈希槽都有节点进行管理的时候，集群处于online 状态。同样的，如果有一个哈希槽没有被管理到，那么集群处于off上面图中4个实例节点组成了一个集群，集群之间的信息通过 Gossip协议 进行交互，这样就可以在某一节点记录其他节点的哈希况。

2.2 为什么需要Cluster模式

单机的吞吐无法承受持续扩增的流量的时候，最好的办法是从横向（scale out）和 纵向（scale up）两方面进行扩展，这个我们务系列 的时候已经讨论过了。

- 纵向扩展（scale up）：将单个实例的硬件资源做提升，比如 CPU核数量、内存容量、SSD容量。

- 横向扩展 (scale out)：横向扩增 Redis 实例数，这样每个节点只负责一部分数据就可以，分担一下压力，典型的分治思维。



那横向扩展和纵向扩展各有什么优缺点呢？

- scale up 虽然操作起来比较简易。但是没法解决Redis一些瓶颈问题，比如持久化（如轮式RDB快照还是AOF指令），遇到大率会很低，响应慢。另外，单台服务器硬件扩容也是有限制的，不可能无限操作。
- scale out 更容易扩展，分片的模式可以解决很多问题，包括单一实例节点的硬件扩容限制、成本限制，还可以分摊压力，精护。但是同时也要面临分布式带来的一些问题

现实情况下，在面对千万级甚至亿级别的流量的时候，很多大厂都是在千百台的实例节点组成的集群上进行流量调度、服务 Cluster模式，是业内广泛采用的模式。

3 Cluster 实现原理

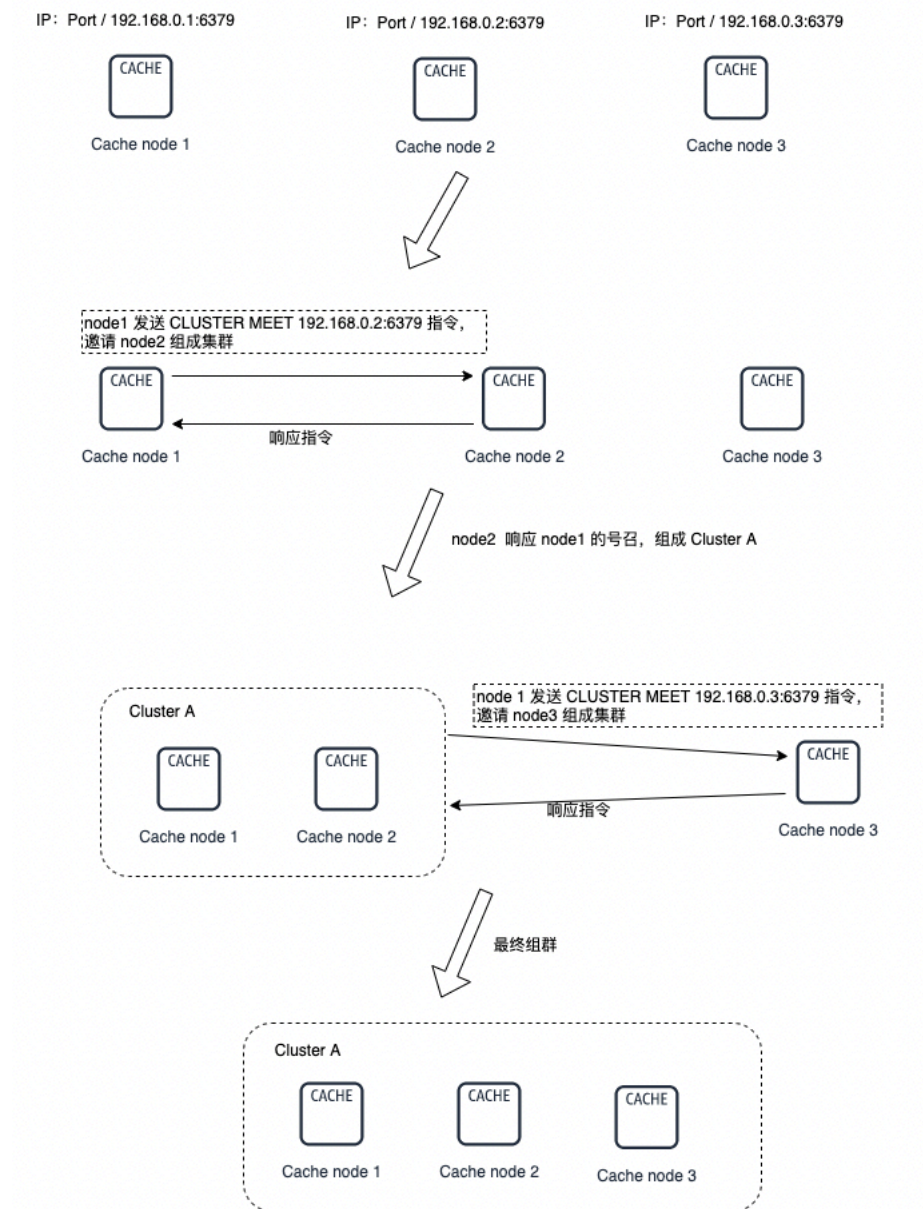
3.1 集群的组群过程

集群是由一个个互相独立的节点 (redis node) 组成的，所以刚开始的时候，他们都是隔离，毫无联系的。我们需要通过一些招起，最终才能组成真正的可协调工作的集群。

各个节点的联通是通过 CLUSTER MEET 命令完成的： `CLUSTER MEET <ip> <port>` 。

具体的做法是其中一个node向另外一个 node (指定 ip 和 port) 发送 CLUSTER MEET 命令，这样就可以让两个节点进行握手成功之后，node 节点就会将握手另一侧的节点添加到当前节点所在的集群中。

这样一步步的将需要聚集的节点都圈入同一个集群中，如下图：



3.2 集群数据分片原理

现在的Redis集群分片的做法，主要是使用了官方提供的 Redis Cluster 方案。这种方案就是的核心就是集群的实例节点与哈希槽映射与管理。下面我们来看看他具体的步骤。

3.2.1 哈希槽（slots）的划分

这个前面已经说过了，我们会将整个Redis数据库划分为16384个哈希槽，你的Redis集群可能有n个实例节点，每个节点可以处理点，这些节点把 16384个槽位瓜分完成。

而你实际存储的Redis键值信息也必然归属于这 16384 个槽的其中一个。slots 与 Redis Key 的映射是通过以下两个步骤完成的：

- 使用 CRC16 算法计算键值对信息的Key，会得出一个 16 bit 的值。
 - 将 第1步中得到的 16 bit 的值对 16384 取模，得到的值会在 0 ~ 16383 之间，映射到对应到哈希槽中。
- 当然，可能在一些特殊的情况下，你想把某些key固定到某个slot上面，也就是同一个实例节点上。这时候可以用hash tag能槽位等于 tag 所在的槽位。
- 其实现方式为在key中加个{}，例如test_key{1}。使用hash tag后客户端在计算key的crc16时，只计算{}中数据。如果没使用{}个key进行crc16计算。下面演示下hash tag使用：

```
127.0.0.1:6380> cluster keyslot user:case{1}
(integer) 1024
127.0.0.1:6380> cluster keyslot user:favor
(integer) 1023
127.0.0.1:6380> cluster keyslot user:info{1}
(integer) 1024
```

如上，使用hash tag 后会对应到通一个hash slot：1024中。

3.2.2 哈希槽（slots）的映射

一种是初始化的时候均匀分配，使用 `cluster create` 创建，会将 16384 个 slots 平均分配在我们的集群实例上，比如你有 n 个节点是 $16384 / n$ 个了。

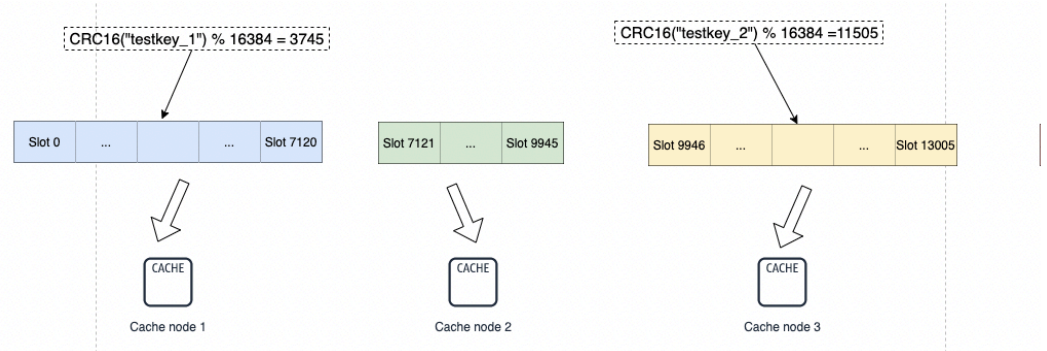
另一种是通过 `CLUSTER MEET` 命令将 `node1`、`node2`、`node3`、`node4` 4 个节点联通成一个集群，刚联通的时候因为还没分配叫状态。我们使用 `cluster addslots` 命令来指定。

指定的好处就是性能好的实例节点可以多分担一些压力。

可以通过 `addslots` 命令指定哈希槽范围，比如下图中，我们哈希槽是这么分配的：实例 1 管理 0 ~ 7120 哈希槽，实例 2 管理 7121 ~ 9945 哈希槽，实例 3 管理 9946 ~ 13005 哈希槽，实例 4 管理 13006 ~ 16383 哈希槽。

```
redis-cli -h 192.168.0.1 -p 6379 cluster addslots 0,7120
redis-cli -h 192.168.0.2 -p 6379 cluster addslots 7121,9945
redis-cli -h 192.168.0.3 -p 6379 cluster addslots 9946,13005
redis-cli -h 192.168.0.4 -p 6379 cluster addslots 13006,16383
```

slots 和 Redis 实例之间的映射关系如下：



key `testkey_1` 和 `testkey_2` 经过 CRC16 计算后再对 slots 的总个数 16384 取模，结果分别匹配到了 `cache1` 和 `cache3` 上

3.3 数据复制过程和故障转移

3.3.1 数据复制

Cluster 是具备 Master 和 Slave 模式，Redis 集群中的每个实例节点都负责一些槽位，比如上图中的四个节点分管了不同的槽位区。需要一个 Slave 节点，Slave 节点是通过《Redis 系列3：高可用之主从架构》方式同步主节点数据。节点之间保持 TCP 通信，当 Redis Cluster 自动会将对应的 Slave 节点选为 Master，来继续提供服务。与纯主从模式不同的是，主从节点之间并没有读写分离，宕机的高可用备份，所以更合理来说应该是主备模式。

如果主节点没有从节点，那么一旦发生故障时，集群将完全处于不可用状态。但也允许配置 `cluster-require-full-coverage` 可用，其他节点正常提供服务，这是为了避免全盘宕机。

主从切换之后，故障恢复的主节点，会转化成新主节点的从节点。这种自愈模式对提高可用性非常有帮助。

3.3.2 故障检测

一个节点认为某个节点宕机不能说明这个节点真的挂起了，无法提供服务了。只有占据多数的实例节点都认为某个节点挂起了，才会进行主从切换的工作。

Redis 集群的节点采用 Gossip 协议来广播信息，每个节点都会定期向其他节点发送 ping 命令，如果接受 ping 消息的节点在指定时间内没有响应，则认为该节点失联了 (PFail)，则发送 ping 的节点就把接受 ping 的节点标记为主观下线。

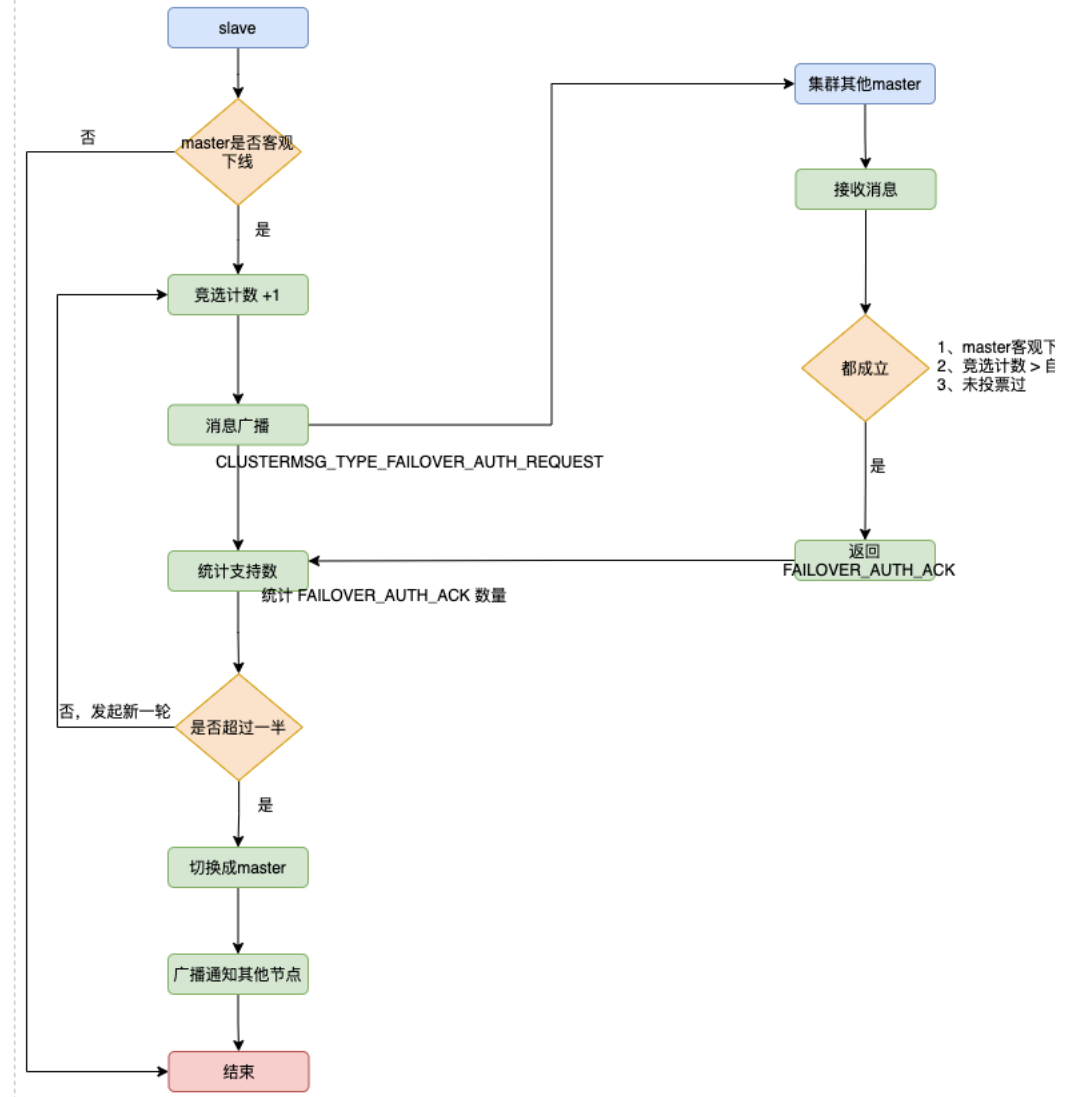
如果集群半数以上的主节点都将主节点 xxx 标记为主观下线，则节点 xxx 将被标记为客观下线，然后向整个集群广播，让其它节点，并立即对下线的节点进行主从切换。

3.3.3 主从故障转移

当一个从节点发现自己正在复制的主节点进入了已下线，则开始对下线主节点进行故障转移，故障转移的步骤如下：

- 如果只有一个 slave 节点，则从节点会执行 `SLAVEOF no one` 命令，成为新的主节点。
- 如果是多个 slave 节点，则采用选举模式进行，竞选出新的 Master
 - 集群中设立一个自增计数器，初始值为 0，每次执行故障转移选举，计数就会 +1。
 - 检测到主节点下线的从节点向集群所有 master 广播一条 `CLUSTERMSG_TYPE_FAILOVER_AUTH_REQUEST` 消息，并携带投票的主节点都向这个从节点投票。
 - 如果收到消息、并具备投票权的主节点未投票给其他从节点（只能投一票哦，所以投过了不行），则返回一条 `CLUSTERMSG_TYPE_FAILOVER_AUTH_ACK` 消息，表示支持。
 - 参与选举的从节点都会接收 `CLUSTERMSG_TYPE_FAILOVER_AUTH_ACK` 消息，如果收集到的选票 大于等于 $(n/2) + 1$ 具备选举权的主节点，那么这个从节点就被选举为新主节点。
 - 如果这一轮从节点都没能争取到足够多的票数，则发起新一轮选举（自增计数器 +1），直至选出新的 master。
- 新的主节点会撤销所有对已下线主节点的 slots 指派，并将这些 slots 全部指派给自己。
- 新的主节点向集群广播一条 `PONG` 消息，这条 `PONG` 消息可以让集群中的其他节点立即知道这个节点已经由从节点变成了主节点已经接管了原本由已下线节点负责处理的槽。
- 新的主节点开始接收和自己负责处理的槽有关的命令请求，故障转移完成。

跟哨兵类似，两者都是基于 Raft 算法来实现的，流程如图所示：



3.4 client 访问 数据集群的过程

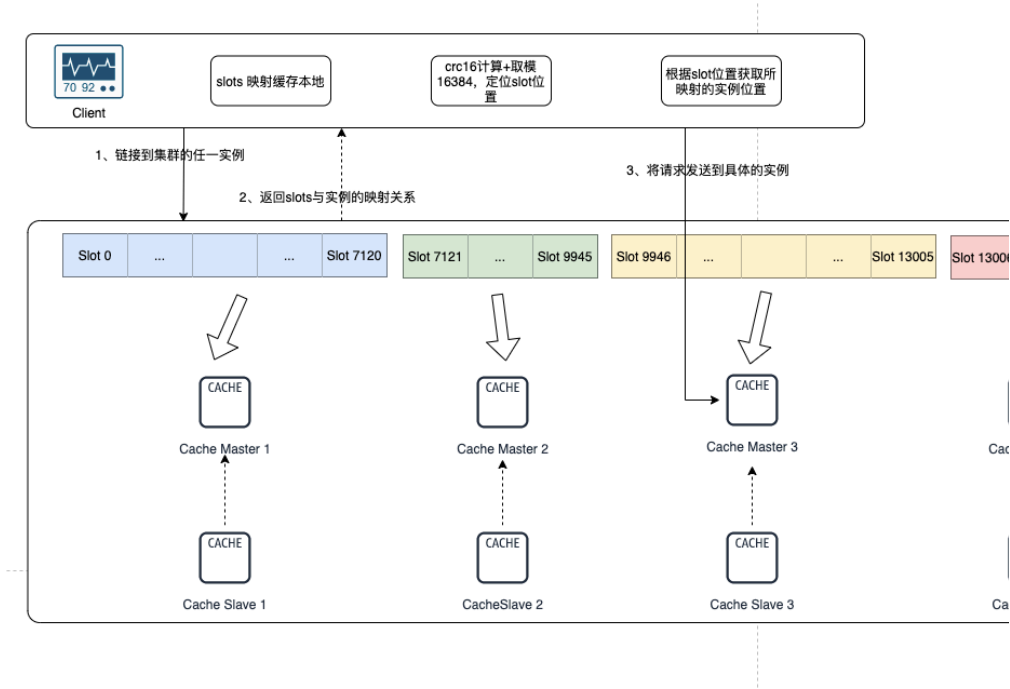
3.4.1 定位数据所在节点

我们前面说过了，Redis 中的每个实例节点会将自己负责的哈希槽信息 通过 Gossip 协议广播给集群中其他的实例，实现了 slots 话，每个实例都知道整个集群的哈希槽分配情况以及映射信息。

所以客户端想要快捷的连接到服务端，并对某个redis数据进行快捷访问，一般是经过以下步骤：

- 客户端连接任一实例，获取到slots与实例节点的映射关系，并将该映射关系的信息缓存在本地。
- 将需要访问的redis信息的key，经过CRC16计算后，再对16384 取模得到对应的 Slot 索引。
- 通过slot的位置进一步定位到具体所在的实例，再将请求发送到对应的实例上。

下图展示了 Redis 客户端如何定位数据所在节点：



4 总结

- 哨兵模式已经实现了故障自动转移的能力，但业务规模的不断扩展，用户量膨胀，并发量持续提升，会出现了 Redis 响应慢
- 使用 Redis Cluster 集群，主要解决了大数据量存储导致的各种慢问题，同时也便于横向拓展。在面对千万级甚至亿级别的
- 做法是在千百台的实例节点组成的集群上进行流量调度、服务治理的。
- 整个Redis数据库划分为16384个哈希槽，Redis集群可能有n个实例节点，每个节点可以处理0个 到至多 16384 个槽点，这!
- 分完成。
- Cluster 是具备Master 和 Slave模式，Redis 集群中的每个实例节点都负责一些槽位，节点之间保持TCP通信，当Master发
- Cluster自动会将对应的Slave节点选为Master，来继续提供服务。
- 客户端能够快捷的连接到服务端，主要是将slots与实例节点的映射关系存储在本地，当需要访问的时候，对key进行CRC16
- 模得到对应的 Slot 索引，再定位到相应的实例上。实现高效的连接。

码字不易，欢迎关注，欢迎转载

作者：Brand

出处：<https://www.cnblogs.com/wzh2010/>

本文采用「CC BY 4.0」知识共享协议进行许可，转载请注明作者及出处。

posted @ 2022-08-05 14:58 Hello-Brand 阅读(9132) 评论(0) 编辑 收藏 举报

会员力量，点亮园子希望